ICLR 2024 | No Training, No Prompts, No Normal Samples: Letting Test Images Score Each Other

MuSc turns a batch of unlabeled industrial images into its own supervision signal for zero-shot anomaly classification and segmentation

Automated visual inspection on a factory line has to answer two questions about every product image at once: is there a defect (anomaly classification, AC), and where exactly is it, down to the pixel (anomaly segmentation, AS)? Two families of methods dominate. One-class methods build a memory bank of normal appearance, which means collecting a large set of defect-free reference images for every product. CLIP-based zero-shot methods skip the reference images but lean on hand-written text prompts instead.

Both assumptions get awkward in practice. Gathering normal samples is a poor fit for new products or short production runs, and writing good prompts is both an art and a losing battle against the sheer variety of real defects. The setting that actually matches a factory floor is stricter: no training, no prompts, and no normal reference images, yet still a pixel-level defect map and an image-level verdict.

MuSc (short for Mutual Scoring), presented at ICLR 2024, is built for exactly that setting. Its central observation is almost mundane: a batch of unlabeled test images already contains enough normal and abnormal evidence to find defects on its own. Normal appearance repeats across images, while defects are rare and idiosyncratic, so if you let the images score one another, the anomalies fall out with no labels required.

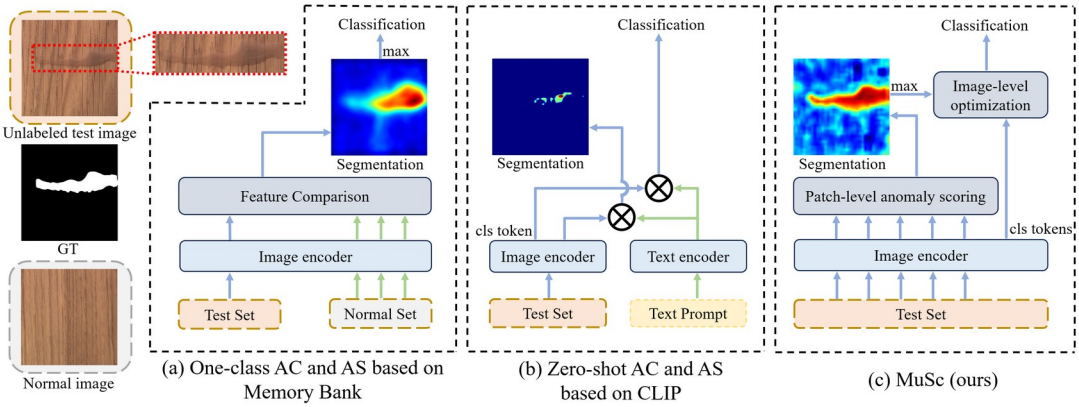

Figure 1. Three paradigms compared. (a) A memory-bank one-class method needs many normal reference images; (b) a CLIP-based zero-shot method relies on extra text prompts; (c) MuSc uses only the unlabeled test images, scoring them at both the patch and image level.

Paper: https://arxiv.org/abs/2401.16753 · Code: https://github.com/xrli-U/MuSc

Why mutual scoring works

The intuition rests on a checkable statistical regularity. Split a batch of same-product test images into patches. A normal patch tends to find many near-identical patches in the other images, because normal texture recurs. An abnormal patch finds only a handful of close matches, because a defect is rare and rarely repeats. That asymmetry, in how many similar neighbors a patch has, is by itself a training-free discriminative signal.

A three-stage pipeline

MuSc extracts features with a frozen vision transformer, ViT-L/14-336 from OpenAI's CLIP, never fine-tuned, and threads them through three modules: Local Neighborhood Aggregation with Multiple Degrees (LNAMD), the Mutual Scoring Mechanism (MSM), and Re-scoring with a Constrained Image-level Neighborhood (RsCIN). The figure below is easiest to read by following the data flow from left to right.

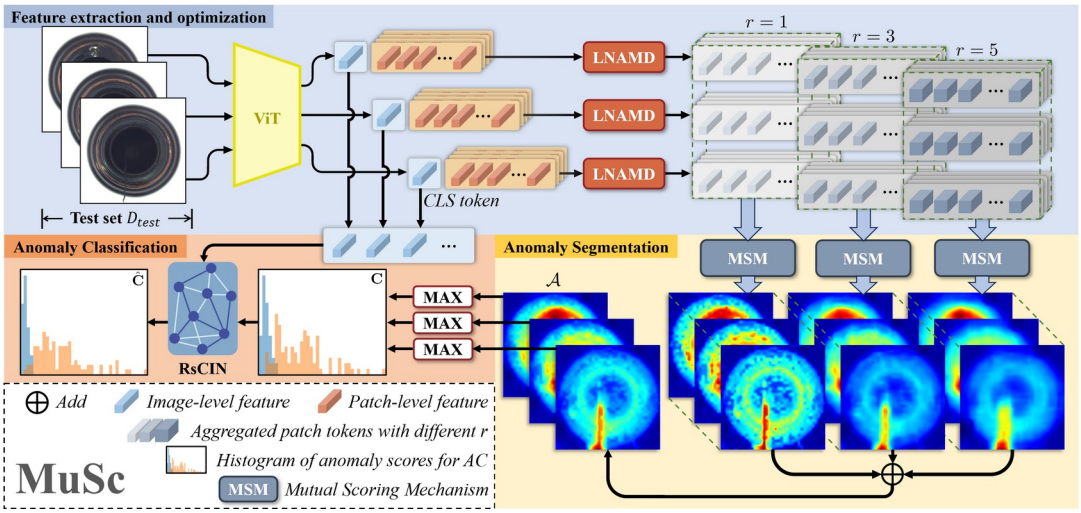

Figure 2. The MuSc architecture. Patch tokens from a frozen ViT are aggregated by LNAMD at several degrees r to represent defects of different sizes; MSM lets the test images score each other to produce the segmentation map; RsCIN then refines the image-level score using the class tokens.

LNAMD comes first. It aggregates each patch token at several spatial scales, using degrees $r \in \{1, 3, 5\}$. Small r captures tiny defects, common on VisA, while large r captures the larger defects common on MVTec AD; combining all three covers a range of defect sizes rather than betting on one.

MSM: images scoring images

MSM is the heart of the method. Each patch's anomaly score is the distance to its nearest match in every other test image. A normal patch has close neighbors everywhere, so that distance stays small; an abnormal patch's stays large. But naively averaging over all images is fragile: an abnormal patch will occasionally land near a similar-looking region in some image, dragging its average down.

MuSc's fix is to keep, for each patch, only the smallest 30% of those mutual scores before averaging. The histograms below show why that helps.

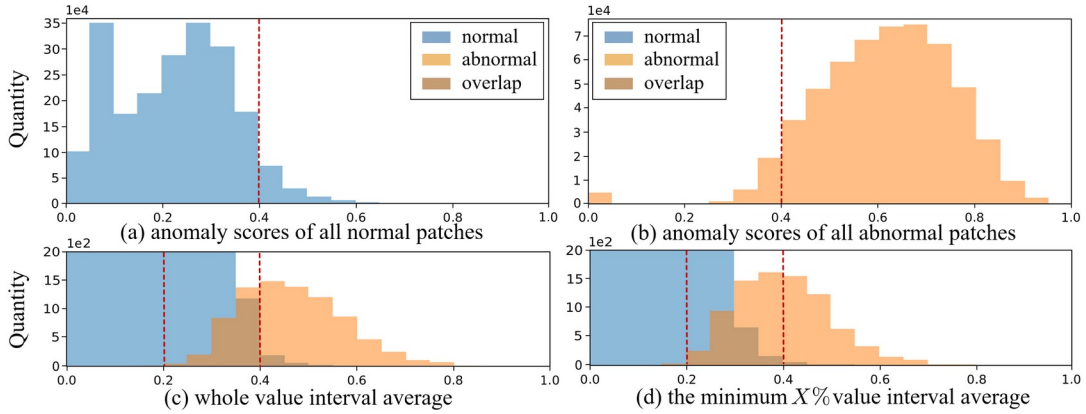

Figure 4. Top: raw nearest-neighbor distances for all normal (a) and abnormal (b) patches. Bottom: scores after averaging over the whole interval (c) versus only the minimum 30% interval (d); blue is normal, orange is abnormal. Restricting to the minimum interval visibly shrinks the overlap between the two.

Compare (c) and (d): averaging only the minimum 30% interval narrows the overlap (the brown region) between the normal and abnormal distributions, so a single threshold separates them far more cleanly. The ablations bear this out: on MVTec AD the "30% + mean" strategy reaches 97.8% image-level AUROC, against just 83.8% for a max-based strategy.

RsCIN: polishing the image-level call

The third stage, RsCIN, targets image-level classification specifically. It builds a constrained neighborhood graph over the class tokens with a multi-window mask, and calls an image normal only when it and its nearest neighbors all look normal, which suppresses the occasional misranked sample.

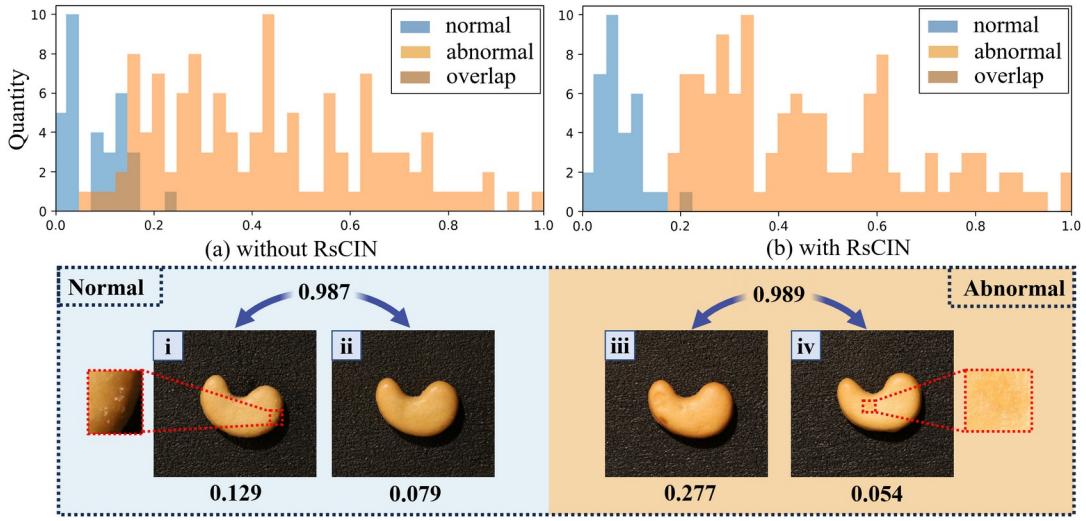

Figure 5. Top: histograms of image-level anomaly scores before (a) and after (b) RsCIN, blue for normal and orange for abnormal. Bottom: a normal and an abnormal re-scoring example. After RsCIN the two classes separate more cleanly.

On VisA, RsCIN lifts image-level AUROC from 90.0% to 92.8%. It is also a plug-in: bolted onto other classification methods, it delivers a consistent boost there too.

Results: zero-shot, yet rivaling supervised methods

On the two standard industrial benchmarks, MVTec AD (15 categories, 10 objects plus 5 textures) and VisA (12 object categories), MuSc sets a new zero-shot state of the art. Classification is scored with AUROC, AP, and F1-max; segmentation adds pixel-level AUROC, F1-max, AP, and Per-Region Overlap (PRO).

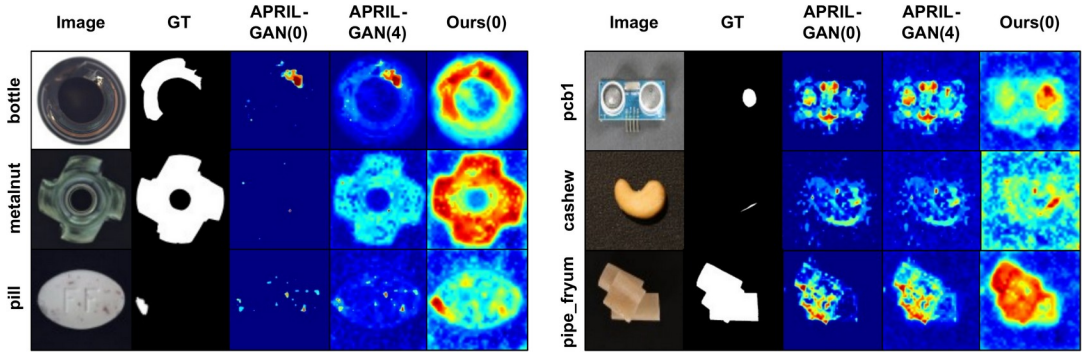

Figure 6. Anomaly-segmentation visualizations on MVTec AD (left) and VisA (right): from left to right, the input image, ground-truth mask, APRIL-GAN at 0-shot and 4-shot, and MuSc at 0-shot. MuSc's heatmaps cover the true defect regions far more completely.

In numbers, MuSc reaches 97.8% image-level AUROC and 97.3% pixel-level AUROC on MVTec AD. On segmentation it beats the second-best zero-shot method by +21.1% PRO, +21.9% AP, and +18.4% F1-max. On VisA it gains more than ten points over the runner-up in both classification and segmentation.

Table 1. Zero-shot comparison on MVTec AD and VisA (%). AUROC-cls is image-level classification, AUROC-segm is pixel-level segmentation, PRO is Per-Region Overlap.

Dataset	Method	AUROC-cls	AUROC-segm	PRO-segm
MVTec AD	WinCLIP	91.8	85.1	64.6
MVTec AD	APRIL-GAN	86.1	87.6	44.0
MVTec AD	MuSc (ours)	97.8	97.3	93.8
VisA	WinCLIP	78.1	79.6	56.8
VisA	APRIL-GAN	78.0	94.2	86.8
VisA	MuSc (ours)	92.8	98.8	92.7

The cross-setting comparison is the striking part. Using no reference images at all, MuSc outperforms most 4-shot methods, beats the 32-shot RegAD, matches the 8-shot GraphCore, and even holds its own against full-shot methods like CutPaste, IGD, and NSA that train on the entire normal set.

Takeaway and limits

MuSc's contribution is to make good use of a fact prior work left on the table: a batch of unlabeled test images can supervise itself. Because normal structure recurs across images

while defects stand alone, mutual scoring pulls them apart, and zero-shot performance climbs to near-supervised levels for both classification and segmentation.

The approach does come with conditions. MuSc needs a batch of same-category test images to run, since a single isolated image has nothing to score against, and the all-pairs comparison carries an inference cost. The paper offers a knob that splits the dataset into s parts to cut per-image latency and memory (splitting into three parts drops per-image time from roughly 999 ms to about 514 ms). Where batched test data is available, MuSc is a practical starting point that asks for no training, no prompts, and no normal samples.

More broadly, MuSc is a reminder that the data you already have can carry more supervision than it first appears. Before reaching for labels, prompts, or a curated reference set, it can be worth asking whether the test batch itself already holds the answer, and simply needs a way to let its images compare notes.